ACADEMIC & TECHNICAL AUDIT REPORT

AI MOCK INTERVIEW 2.0

Comprehensive Module Analysis, Actual Database Verification, and Source-Code Evidentiary Architectural Audit

Project Name:	**MAPD AI Mock Interview 2.0**
Repository:	**AI-Mock-Interview-2.0-main**
Audit Scope:	**8 System Modules & Actual Storage Providers**
Audit Methodology:	**Read-Only Static Code & Config Inspection**
Primary Framework:	**Next.js 15.4.10 / React 19.1.0 / TypeScript**
Primary Database:	**Convex Serverless Document DB (v1.25.4)**
Auth Provider:	**Clerk (v6.39.7)**
Audit Date:	**October 2026**

1. Project Overview

MAPD AI Mock Interview 2.0 is a full-stack web application architected using **Next.js 15 (App Router)**, **React 19**, and **TypeScript**. The system facilitates end-to-end technical interview simulations for software engineering candidates across diverse roles and framework specializations.

Unlike conventional mock interview platforms that rely solely on static questionnaires or mock static landing pages, this project integrates live speech interactions, adaptive dynamic follow-up questioning, real-time in-browser computer vision posture tracking, and multi-cloud video recording persistence.

> **Audit Purpose & Grounding:** This document serves as an exhaustive, evidence-based technical reference for academic evaluations, project vivas, and architectural defense. Every claim in this document is directly supported by citations of executable source files, initialized SDKs, and database schema declarations.

2. Comprehensive Explanation of the Eight Modules

Module 1: User Authentication Module

Authentication is managed exclusively by **Clerk** (`@clerk/nextjs`, v6.39.7). Clerk provides managed user authentication, session tokens (JWTs), OAuth provider integrations, and edge route middleware protection.

- **Protected Routes:** Configured in `middleware.ts` using `clerkMiddleware` and `createRouteMatcher`. Public routes are `/`, `/sign-in(.*)`, `/sign-up(.*)`, `/questions(.*)`, `/upgrade(.*)`, and `/how-it-works(.*)`. All other routes require `await auth.protect()`.
- **API Endpoint Security:** Server routes invoke `await currentUser()` or `await auth()` from `@clerk/nextjs/server`, returning HTTP 401 Unauthorized if no active user session exists.
- **User Profile Persistence:** User credentials remain in Clerk's secure cloud, while profile details (name, email, imageUrl) are mirrored into Convex database table `UserTable` via the mutation `api.users.CreateNewUser` invoked in `app/Provider.tsx` upon login.
- **Evidence Files:** `middleware.ts` (Lines 1-26), `app/layout.tsx` (Lines 33-44), `app/Provider.tsx` (Lines 8-32), `convex/users.ts` (Lines 4-34), `convex/schema.ts` (Lines 5-9).

Module 2: Interview Setup Module

Candidates configure their mock interview using a client-side modal dialog (`CreateInterviewDialog.tsx`).

- **Configuration Fields:**
  - `jobTitle`: Job Role / Position (e.g., "Senior Java Developer") [Mandatory text].
  - `techStack`: Targeted frameworks / tools (e.g., "Spring Boot, SQL") [Optional text].
  - `experienceLevel`: Select dropdown ("Fresher", "0–1 years", "1–2 years" [default], "2–3 years", "3+ years (Senior)").
  - `interviewerGender`: AI Persona toggle ("male" - Alex Vance, or "female" - Sarah Jenkins [default]).
  - `jobDescription`: Detailed role requirements [Mandatory textarea].
  - `file`: Optional resume PDF upload (MIME `application/pdf`, max 10MB).

- **Validation & Creation:** Validates role and description strings, sends multipart FormData to `POST /api/generate-interview-questions`, and persists the created session via Convex mutation `api.Interview.SaveInterviewQuestion` with initial status `'draft'`.
- **Evidence Files:** `app/(routes)/_components/CreateInterviewDialog.tsx` (Lines 23-167), `convex/Interview.ts` (Lines 4-29), `convex/schema.ts` (Lines 11-31).

Module 3: Resume Upload Module

The resume upload module provides optional resume attachment during interview creation.

- **Supported Format: PDF only** (`application/pdf`), enforced in `CreateInterviewDialog.tsx` (Line 46) and size limit ≤ 10 MB.
- **Actual Storage Service: ImageKit** (`imagekit` SDK v6.0.0). Verified in `app/api/generate-interview-questions/route.tsx` (Lines 47-67). Uploads via `imagekit.upload()` and stores the returned CDN URL in Convex `InterviewSessionTable.resumeUrl`.
- **Fallback Storage: None.** If ImageKit credentials are unconfigured or dummy, `resumeUrl` remains empty. Convex/Supabase fallbacks exist exclusively for video recordings.
- **Content Parsing Status: Not implemented in executable code.** The PDF is uploaded to ImageKit, but no text extraction (OCR, `pdf-parse`) is performed. Questions are generated from user-entered form text.
- **Deletion Status: Not implemented.** No endpoint exists to delete resume PDFs from ImageKit.
- **Evidence Files:** `CreateInterviewDialog.tsx` (Lines 42-68), `app/api/generate-interview-questions/route.tsx` (Lines 47-67).

Module 4: AI Question Generation Module

Questions are generated through a dual-engine mechanism:

- **Initial 8-Phase Question Generation:** Executed deterministically by `generateRoleAwareQuestions()` in `lib/roleInterviewerEngine.ts`. The role is mapped to 13 domains (Java, Python, Frontend, Backend, Fullstack, AI/ML, Data Science, Analyst, DevOps, Cloud, QA, Cybersecurity, Software Engineer). Constructs 8 structured phases:
  1. Introduction (Easy)
  2. Role Fundamentals (Easy)
  3. Intermediate Concepts (Medium)
  4. Practical & Programming Logic (Medium)
  5. Project Discussion (Medium)
  6. Resume & Stack Deep-Dive (Medium)
  7. Problem Solving & Latency Scenarios (Hard)
  8. HR & Behavioral STAR (Easy)
- **Dynamic Adaptive Follow-Up (LLM):** Executed by `POST /api/generate-followup-question` calling `generateAdaptiveFollowUp()`.
  - **Model Configured: Google Gemini 1.5 Flash** (`gemini-1.5-flash`) via REST endpoint: `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${apiKey}`.
  - **Fallback:** If Gemini is unconfigured or times out (7000ms), executes rule-based pivoting when candidates struggle ("I don't know") or keyword-probing for Spring, JWT, React, SQL, Docker, Pandas, PyTorch, etc.
- **Evidence Files:** `lib/roleInterviewerEngine.ts` (Lines 44-531), `app/api/generate-followup-question/route.tsx` (Lines 4-30), `convex/Interview.ts` (Lines 4-29).

Module 5: Interactive Interview Session Module

Implemented in `app/(routes)/interview/[interviewId]/start/page.tsx`, handling live interaction, hardware capture, and recording.

- **Media Hardware:** Connects webcam and microphone via `navigator.mediaDevices.getUserMedia()`. Gracefully degrades to text mode if hardware is blocked.
- **Voice Synthesis & Recognition:** Native browser **Web Speech API**. Text-to-Speech (TTS) uses `window.speechSynthesis` with customized pitch (0.9 for male Alex Vance, 1.1 for female Sarah Jenkins). Speech-to-Text (STT) uses `webkitSpeechRecognition` for continuous voice transcription.
- **Computer Vision Telemetry:** Implemented in `utils/facialAnalysis.ts`. Every 1200ms, an in-memory Canvas 2D samples webcam pixels ($320 imes 240$), computing perceived luminance and skin-color centroid coordinates to estimate eye contact ($65\%-98\%$), head posture ("Centered", "Slight Tilt", "Looking Away"), and composure.
- **Video Recording & Multi-Tier Storage:** MediaRecorder captures `video/webm`. When the interview ends:
  1. **Tier 0:** Instantly buffers to browser **IndexedDB** (`MAPD_Interview_Recordings_DB`).
  2. **Tier 1:** Uploads directly to **Convex Cloud Object Storage** via signed URL.
  3. **Tier 2 (Fallback):** Uploads to **Supabase Storage** bucket `interview-recordings` (if configured).
  4. **Tier 3 (Fallback):** Uploads to **ImageKit** folder `/recordings/${userId}`.
  5. **Tier 4 (Fallback):** Writes directly to server local disk at `public/uploads/recordings/${fileName}`.

- **Evidence Files:** `start/page.tsx` (Lines 1-1219), `utils/facialAnalysis.ts` (Lines 1-137), `utils/recordingStorage.ts` (Lines 1-287), `app/api/upload-recording/route.ts` (Lines 1-165).

Module 6: Response Evaluation and Scoring Module

Evaluates candidate spoken answers at `POST /api/interview-feedback`.

- **Evaluation Logic: Deterministic rule-based evaluation** (`evaluateCandidateAnswers`). The feedback endpoint does not call Gemini LLM, ensuring zero hallucinations.
- **Criteria:**
  - Word count scoring: 0 words = 3.0/10; <15 words = 5.5/10; ≥15 words = 8.5/10 with keyword depth checks.
  - Filler words: Regex scan `/\b(um|uh|like|you know|basically|actually|sort of|kind of)\b/gi`.
  - Speaking Pace: Calculated in Words Per Minute (WPM).
  - Weighted formula: `Rating = (Technical × 35%) + (ProblemSolving × 25%) + (Relevance × 15%) + (Completeness × 15%) + (Communication × 10%)`.
  - STAR answers: Generates Situation-Task-Action-Result model answers for each question.
- **Persistence:** Saves to Convex DB table `InterviewSessionTable.feedback` via `UpdateFeedback` mutation.
- **Evidence Files:** `app/api/interview-feedback/route.tsx` (Lines 1-326), `convex/Interview.ts` (Lines 70-105).

Module 7: Performance Dashboard Module

Provides analytics and interview session history in `app/(routes)/dashboard/page.tsx`.

- **Metrics:** Displays Total Sessions, Completed Sessions, Average Score, Practice Hours (Total Sessions × 0.5 hrs), score trajectory bar chart, and filterable interview cards.
- **Data Source Verification:**
  - **Live Backend Data:** Query `api.Interview.GetInterviewList` queries Convex DB in real-time.
  - **Hardcoded Elements:** Aggregated Skill Competency progress bars in `ProgressAnalytics.tsx` use static values (8.2, 7.8, 8.5, 8.4).
  - **Local Storage:** Bookmarked flashcards dialog reads from `localStorage.getItem("mapd_saved_flashcards")`.
- **Evidence Files:** `dashboard/page.tsx` (Lines 1-222), `ProgressAnalytics.tsx` (Lines 1-124), `InterviewCard.tsx` (Lines 1-233).

Module 8: Database and Session Management Module

Carefully verified through source code and dependencies:

- **Primary Database: Convex** (v1.25.4) serverless reactive document database. Stores `UserTable` and `InterviewSessionTable`.
- **Supabase PostgreSQL Verification: NOT USED.** No SQL schemas, migrations, or tables exist in Supabase. Supabase is referenced in code solely for optional video file storage.
- **Convex Dual Role:** Convex serves as both the primary reactive document database and the serverless TypeScript query/mutation execution platform.
- **Session Cleanup:** `InterviewCard.tsx` invokes `DeleteInterview` mutation to delete records from Convex and calls `/api/delete-recording` to unlink local disk files or delete from cloud storage.
- **Evidence Files:** `convex/schema.ts` (Lines 1-32), `convex/Interview.ts` (Lines 1-243), `convex/users.ts` (Lines 1-34).

3. Module-Wise Input, Processing, and Output Details

Module	Inputs	Processing Logic	Outputs / Side Effects	Key Source
1. Authentication	OAuth credentials, Clerk session tokens	Clerk edge route interception, session token verification, Convex user record synchronization	Active session cookie, mirror record in Convex `UserTable`	`middleware.ts`, `users.ts`
2. Setup	Role, tech stack, experience, persona, PDF file	Client validation, user resolution, dispatching generation request, session creation	Convex `InterviewSessionTable` record (`status: 'draft'`)	`CreateInterviewDialog.tsx`
3. Resume Upload	PDF file buffer (≤ 10 MB)	ImageKit API binary upload call, URL capture	Public CDN URL stored in `resumeUrl`	`route.tsx` (questions)
4. Question Gen	Role, stack, experience strings, answers	13-domain role normalization, 8-phase rule engine, Google Gemini 1.5 Flash adaptive follow-ups	8-phase Question array, dynamic follow-up JSON	`roleInterviewerEngine.ts`
5. Live Session	Webcam video stream, mic audio stream	Web Speech TTS/STT, Canvas 2D skin centroid sampling, MediaRecorder video chunking	Live dialogue transcript, real-time composure metrics, WebM video blob	`start/page.tsx`, `facialAnalysis.ts`
6. Evaluation	Spoken transcripts, questions, duration	Deterministic transcript word/keyword matching, filler word regex count, WPM calculation	Weighted score (1-10), filler analytics, STAR model feedback	`interview-feedback/route.tsx`
7. Dashboard	Convex User ID (`uid`)	Reactive Convex database queries, score averaging, client-side filtering	Total/completed metrics, session cards, score progression chart	`dashboard/page.tsx`
8. DB & Storage	Session metadata, video blobs	Convex schema validation, 4-tier storage pipeline, cascade deletion	Persistent document records, cloud/disk media storage	`convex/schema.ts`, `recordingStorage.ts`

4. Verified Technology Stack Table

Component	Actual Technology / Service	Purpose	Source Evidence	Status
Framework	Next.js 15.4.10 (App Router)	Full-stack application architecture, SSR, API routes	`package.json`, `app/layout.tsx`	Verified

Component	Actual Technology / Service	Purpose	Source Evidence	Status
Frontend Library	React 19.1.0	UI component hierarchy and state hooks	`package.json`	Verified
Language	TypeScript ^5	Static type safety across components and schemas	`tsconfig.json`	Verified
Styling	Tailwind CSS v4 & Radix UI	Utility-first styles, dark mode, dialog/tabs	`globals.css`, `tailwind.config.ts`	Verified
Authentication	Clerk (`@clerk/nextjs` ^6.39.7)	Identity, OAuth, session JWTs, edge middleware	`middleware.ts`, `app/Provider.tsx`	Verified
Primary Database	Convex Serverless DB (^1.25.4)	Reactive document database & cloud functions	`convex/schema.ts`, `Interview.ts`	Verified
Offline Client DB	HTML5 IndexedDB	Zero-loss client video buffering before upload	`utils/recordingStorage.ts`	Verified
AI LLM Engine	Google Gemini 1.5 Flash	Dynamic adaptive interviewer follow-ups	`lib/roleInterviewerEngine.ts`	Verified
Resume Storage	ImageKit (`imagekit` ^6.0.0)	Cloud media CDN for uploaded resume PDFs	`generate-interview-questions/route.tsx`	Verified
Video Storage (Primary)	Convex Cloud Object Storage	Direct client binary upload bypassing serverless limits	`convex/Interview.ts` (GenerateUploadUrl)	Verified
Video Storage (Cloud 1)	Supabase Storage (Bucket)	Secondary fallback (bucket `interview-recordings`)	`upload-recording/route.ts`	Partially Verified
Video Storage (Cloud 2)	ImageKit Object Storage	Tertiary fallback for recorded videos	`upload-recording/route.ts`	Verified
Video Storage (Disk)	Local Server Filesystem	Quaternary fallback (`public/uploads/recordings`)	`upload-recording/route.ts` (5 files present)	Verified
Voice Interaction	Native Web Speech API (TTS/STT)	Browser voice reading and candidate dictation	`start/page.tsx`, `interviewerConfig.ts`	Verified
Vision Telemetry	Native Canvas 2D Heuristics	Skin centroid posture and eye contact tracking	`utils/facialAnalysis.ts`	Verified
Rate Limiting	Arcjet (`@arcjet/next`)	Token bucket rate limiting on question generation	`utils/arcjet.ts`, `generate-interview-questions`	Verified
Streaming Avatar	Akool Avatar SDK	Live avatar streaming SDK & API routes	`package.json`, `akool-session/route.ts`	Orphaned in UI

5. Database Architecture and Data Persistence

The sole operational database is **Convex**, an event-driven serverless document database. Traditional SQL engines (PostgreSQL, MySQL) are absent from the executable stack.

Data Model Schemas (`convex/schema.ts`)

```
1. UserTable:
   - _id: Id<'UserTable'> (Primary Key, Auto-generated)
   - name: string (Candidate Full Name from Clerk)
   - email: string (Unique Candidate Email Address)
   - imageUrl: string (Avatar Profile URL)

2. InterviewSessionTable:
   - _id: Id<'InterviewSessionTable'> (Primary Key, Auto-generated)
   - userId: Id<'UserTable'> (Foreign Key referencing UserTable)
   - status: string ('draft' | 'complete')
   - jobTitle: string | null
   - jobDescription: string | null
   - experienceLevel: optional(string | null)
   - techStack: optional(string | null)
   - interviewerGender: optional(string | null) ('male' | 'female')
   - interviewQuestions: array of QuestionItem { question, answer, category, phase }
   - candidateAnswers: optional array of { question, answer, timestampSeconds }
   - questionTimestamps: optional array of { questionIndex, formattedTime }
   - feedback: optional complete Evaluation Report JSON
   - weakTopics: optional array of string knowledge gaps
   - resumeUrl: string | null (ImageKit CDN URL)
   - recordingUrl: optional string | null (Convex / Disk / Cloud URL)
   - recordingId: optional string | null (Storage ID)
   - storageProvider: optional string ('convex' | 'supabase' | 'imagekit' | 'server')
   - fileSize: optional number (bytes)
   - durationSeconds: optional number (seconds)
```

6. File and Media Storage Architecture

Storage handling is segregated between resume documents and session video recordings:

- **Resumes (PDFs):** Uploaded directly via `imagekit.upload()` in `app/api/generate-interview-questions/route.tsx`. Stores public URL in `InterviewSessionTable.resumeUrl`. No local fallback or deletion mechanism exists for resumes.
- **Video Recordings (.webm):** Managed via a four-tier resilient fallback hierarchy:
  1. **Browser IndexedDB Buffer:** Saved to `MAPD_Interview_Recordings_DB` before network transmission.
  2. **Convex Cloud Object Storage:** Primary cloud destination via direct signed binary upload (`generateUploadUrl`).
  3. **Supabase Storage:** Fallback to bucket `interview-recordings` under `${userId}/${fileName}`.
  4. **ImageKit Object Storage:** Secondary cloud fallback to folder `/recordings/${userId}`.
  5. **Local Server Disk:** Quaternary fallback writing to `public/uploads/recordings/${fileName}` using Node.js `fs.writeFileSync`. (5 actual active files verified on disk).

7. Interview Session Lifecycle and Data Flow

1. **Setup:** Candidate completes setup dialog; PDF resume uploaded to ImageKit; initial 8-phase questions generated via `lib/roleInterviewerEngine.ts`; record saved to Convex (`status: 'draft'`).
2. **Readiness Check:** Candidate validates camera/mic in `/interview/[id]` and selects interviewer persona.
3. **Live Execution:** Candidate starts session in `/interview/[id]/start`; MediaRecorder captures video; Web Speech API synthesizes spoken questions; Canvas 2D tracks posture/eye contact; STT captures candidate voice; Gemini 1.5 Flash generates adaptive follow-up probes.
4. **Conclusion:** MediaRecorder stops; video blob is buffered in IndexedDB and uploaded via the 4-tier pipeline; `/api/interview-feedback` evaluates transcripts; Convex record updated to `status: 'complete'`.
5. **Review & Feedback:** Candidate inspects radar chart and STAR answers in `/interview/[id]/feedback`, reviews recorded video in `/interview/[id]/playback`, or generates targeted practice for weak areas.

8. Text-Based System Architecture Diagram



9. Feature Verification Table

Feature Description	Status	Implementation Details & Evidence
Clerk User Authentication	Implemented	Clerk components in `app/(auth)/*`, middleware in `middleware.ts`, Convex sync in `convex/users.ts`.
Interview Setup Dialog	Implemented	Modal configuration in `CreateInterviewDialog.tsx` with role, stack, experience, and persona.
ImageKit Resume Upload	Implemented	PDF upload to ImageKit via `imagekit.upload()` in `generate-interview-questions/route.tsx`.
Resume PDF Content Parsing	Not Implemented	No text parsing libraries (OCR, `pdf-parse`) exist; questions are derived solely from form fields.

Feature Description	Status	Implementation Details & Evidence
Resume Cloud Deletion	Not Implemented	No endpoint or function exists to delete resume PDFs from ImageKit storage.
8-Phase Question Generation	Implemented	Deterministic 13-domain progression compiled via `lib/roleInterviewerEngine.ts`.
Gemini 1.5 Flash Follow-ups	Implemented	Live adaptive follow-ups via Gemini 1.5 Flash REST API in `generate-followup-question/route.tsx`.
Speech Synthesis & Recog (TTS/STT)	Implemented	Browser native Web Speech API with gender-matching voice selection in `start/page.tsx`.
Canvas 2D Vision Telemetry	Implemented	Real-time skin centroid pixel heuristic calculation in `utils/facialAnalysis.ts` (no external ML models).
IndexedDB Offline Video Buffer	Implemented	Local video buffering in `MAPD_Interview_Recordings_DB` in `utils/recordingStorage.ts`.
Convex Cloud Object Storage	Implemented	Direct binary upload via `GenerateUploadUrl` in `convex/Interview.ts` and `recordingStorage.ts`.
Supabase Bucket Storage	Partially Verified	Upload logic written in `upload-recording/route.ts`, but unconfigured in `.env.local`.
Server Disk Video Storage	Implemented	Fallback in `upload-recording/route.ts`; 5 active `.webm` files confirmed on disk.
Spoken Transcript Evaluation	Implemented	Deterministic scoring, filler word counts, WPM, and STAR feedback in `interview-feedback/route.tsx`.
Reactive Performance Dashboard	Partially Verified	Session metrics & bar charts query live Convex DB; skill competency progress bars use hardcoded values.
Video Playback with Timestamps	Implemented	Custom player with timestamp navigation in `interview/[interviewId]/playback/page.tsx`.
Practice Weak Areas Auto-Gen	Implemented	Generates targeted session from `weakTopics` array in `feedback/page.tsx`.
Bookmarked Flashcards	Implemented	Revision cards stored in client `localStorage` via `SavedFlashcards.tsx`.
Akool Live Streaming Avatar	Orphaned in UI	SDK and API routes exist in codebase, but are never invoked in any user-facing UI component.
Pro Stripe Payment Checkout	Not Implemented	Static coming-soon placeholder card in `app/(routes)/upgrade/page.tsx`.

10. Conclusion

The static source code verification of the **AI Mock Interview 2.0** codebase confirms the actual architectural foundation of the project:

- **Database:** The sole primary operational database is **Convex** (v1.25.4). **Supabase PostgreSQL is not used.**
- **Authentication:** Managed exclusively by **Clerk** (v6.39.7), with profile metadata mirrored into Convex `UserTable`.
- **Artificial Intelligence:** Dynamic LLM generation uses **Google Gemini 1.5 Flash** via HTTP REST API for real-time follow-ups. Initial 8-phase questions and post-interview transcript evaluations are executed by custom **deterministic rule-based engines**.
- **Media Storage:** Resumes are stored on **ImageKit** without PDF text extraction. Video recordings use a resilient **four-tier fallback** (IndexedDB → Convex Storage → Supabase → ImageKit → Local Disk), with active disk files verified in the repository.
- **Telemetry & Voice:** Face composure and eye contact tracking are achieved through native **Canvas 2D skin centroid pixel heuristics**, and speech synthesis/recognition is performed directly via browser **Web Speech API**.

This verified architecture ensures predictable performance, zero-cost client-side computer vision telemetry, high offline recording resilience, and transcript-grounded scoring integrity.